CSE 816 - Software Production Engineering, Mini-Project

End-to-End CI/CD Pipeline for a Calculator REST API

Praneeth Marudi - IMT2023555

GitHub: `https://github.com/praneethem/Calculator_devops`
Docker Hub: `https://hub.docker.com/r/praneethem/calculator-api`

1 Introduction

This project implements a Calculator REST API using Java, Spring Boot, and Maven. Its main objective is to automate the software delivery process using a CI/CD pipeline, reducing manual effort in building, testing, and deployment. The system integrates GitHub, Jenkins, Docker, and Docker Compose to automate delivery, while Nginx provides access to the deployed API. Application health checks and email notifications help verify deployment status and communicate build results.

2 Pipeline Architecture

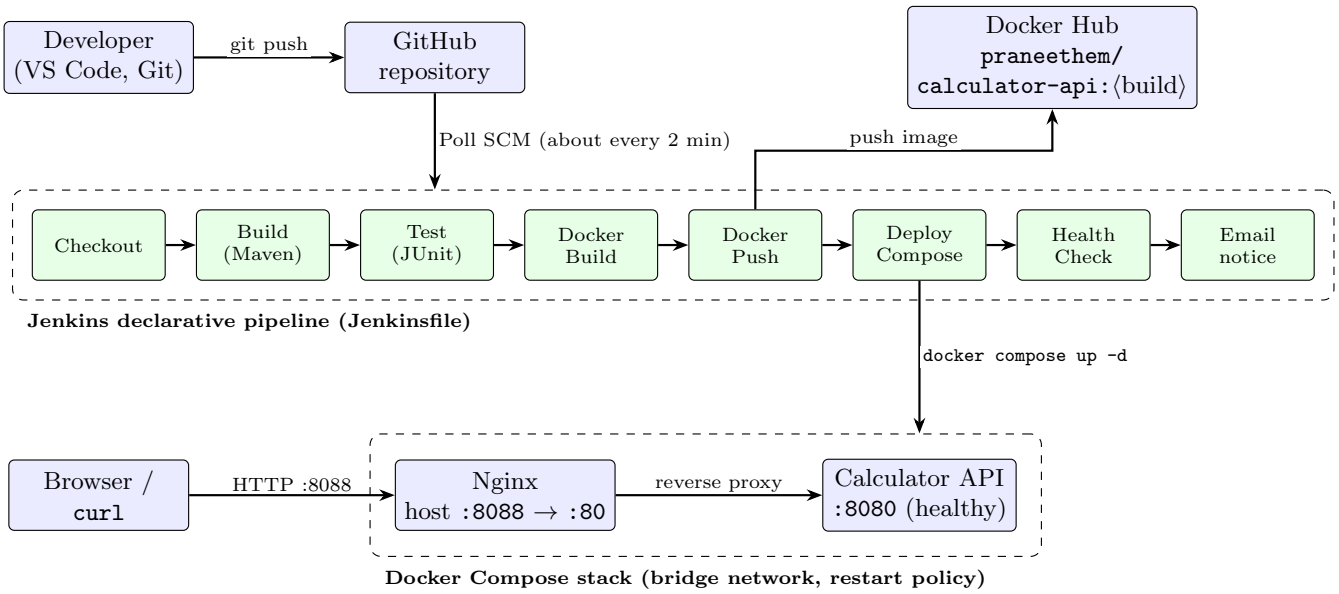


Figure 1: Pipeline and deployment architecture.

Flow. Jenkins periodically checks GitHub for code changes using Poll SCM (approximately every two minutes). When a change is detected, it executes the pipeline stages in sequence, stopping if a stage fails. After successful testing, the Docker image is published to Docker Hub and deployed using Docker Compose. The deployment consists of the Calculator API and Nginx containers connected through a bridge network. Nginx exposes port 8088 on the host and forwards requests to the API on port 8080. Jenkins runs separately on the host at port 8080 and sends email notifications about the build result.

3 Application and Testing

Endpoint (GET)	Behaviour
`/api/v1/add?a=10&b=20`	Returns the sum as JSON containing the operation, inputs, and result.
`/api/v1/subtract?a=20&b=5`	Returns the difference as JSON.
`/api/v1/multiply?a=6&b=7`	Returns the product as JSON.
`/api/v1/divide?a=20&b=4`	Returns the quotient as JSON; division by zero returns HTTP 400.
`/actuator/health`	Returns the application health status; used by Docker Compose and Jenkins.

Missing, non-numeric, or non-finite parameters and division by zero return HTTP 400 with a structured error response, for example `{"status":400,"message":"Division by zero is not allowed","error":"Bad Request"}`. A central exception handler provides consistent error responses across the API.

The test suite contains **16 tests**: one application-context test, five service unit tests, and ten MockMvc controller tests. These cover the arithmetic operations, division by zero, and missing or invalid inputs. Maven Surefire generates JUnit XML reports in `target/surefire-reports/`, which Jenkins publishes. If a test fails, the pipeline stops before Docker image publishing and deployment.

4 CI/CD Stages

Stage	Description
Checkout	Checks out the code from GitHub (job: `Calculator-CI-CD`, branch `main`, script path `Jenkinsfile`).
Build	Runs `./mvnw -B -DskipTests clean package` to compile the application and produce the executable JAR.
Test	Runs `./mvnw -B test`, publishes JUnit results, and fails the build if tests fail.
Docker Build	Builds a multi-stage Docker image tagged `praneethem/calculator-api:<BUILD_NUMBER>`.
Docker Push	Uses the Jenkins credential `dockerhub-creds` to log in with `--password-stdin`, pushes the image to Docker Hub, and logs out.
Deploy Compose	Pulls the new image and runs `docker compose up -d --no-build --remove-orphans` to deploy the updated stack.
Health Check	Checks `http://localhost:8088/actuator/health` up to 30 times, with a 2 s delay between retries. If the check fails, it prints container status and logs and fails the build.
Notifications	Sends success or failure emails containing the job, build number, status, and build URL through Gmail SMTP.

Trigger and safety. Jenkins uses **Poll SCM** (approximately every two minutes), as permitted by the specification. Concurrent builds are disabled, and each Docker image is tagged with its Jenkins build number. Docker Compose updates the deployment as needed. Build #6 was intentionally failed to verify failure detection and email notification. Automatic triggering was tested through multiple commits, including a final verification after restarting Jenkins; Build #15 completed successfully after detecting an SCM change. Docker Hub and SMTP secrets are stored in Jenkins credentials and are not committed to the repository.

5 Containerization and Deployment

Dockerfile. A multi-stage build is used. The first stage uses a Maven and Eclipse Temurin JDK 21 image, copies `pom.xml` first, and prefetches dependencies to enable layer caching before compiling the JAR. The second stage uses a Temurin JRE 21 image, copies only the JAR, runs the application as a non-root user (UID 10001), and declares `EXPOSE 8080`. A `.dockerignore` excludes `.git`, `target`, screenshots, IDE files, logs, and `.env` from the build context. The published Docker image, `praneethem/calculator-api:15`, is 114.7 MB on Docker Hub.

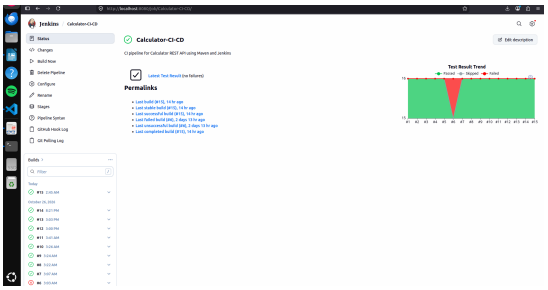
Docker Compose. The `docker-compose.yml` configuration defines the Calculator API service, which uses the image from Docker Hub, checks health through `/actuator/health`, uses `restart: unless-stopped`, and receives configuration through environment variables. It also defines an Nginx (`nginx:1.28-alpine`) reverse proxy on the same bridge network. Nginx waits for the API to become healthy before starting. The command `docker compose up -d` starts the services in the background, while Jenkins pulls and deploys the updated image during the CI/CD pipeline.

6 Results

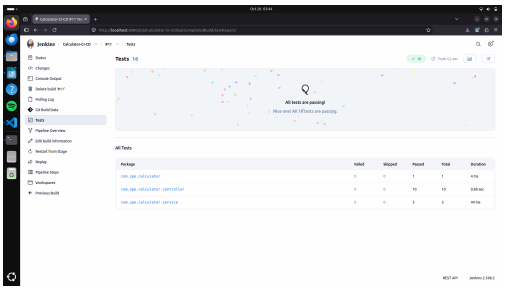
Selected build results:

Build	Result	Notes
#5	Success	Successful run triggered by an SCM change; success email received.
#6	Failure	A deliberately incorrect assertion (999.0 instead of 15.0) caused the Test stage to fail. No image was pushed or deployed, and a failure email was received.
#7	Success	Successful run after restoring the assertion; image `praneethem/calculator-api:7` was produced.
#8	Success	Successful build from `main` (commit `429f005`) after PR #4 was merged.
#11	Success	All 16 tests passed in approximately 4.2 seconds; the success email referenced image `praneethem/calculator-api:11`.
#12	Success	Automatically triggered by an SCM change (commit `7a7ca69`); deployment health check passed.
#15	Success	Automatically triggered after an SCM change following a Jenkins restart; deployment health check passed and a success email was sent.

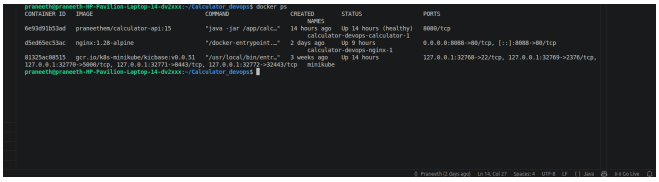
Deployment verification. Docker Compose started both containers. The `docker ps` output showed the Calculator API container using image `praneethem/calculator-api:15` in a healthy state and Nginx publishing `0.0.0.0:8088->80/tcp`. Through Nginx, `/api/v1/add?a=10&b=20` returned HTTP 200 with result 30.0, `/api/v1/divide?a=10&b=0` returned HTTP 400 with a structured JSON error, and `/actuator/health` returned HTTP 200 with status `UP`.



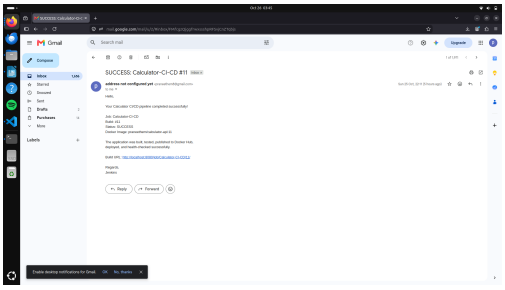
(a) Green Jenkins pipeline



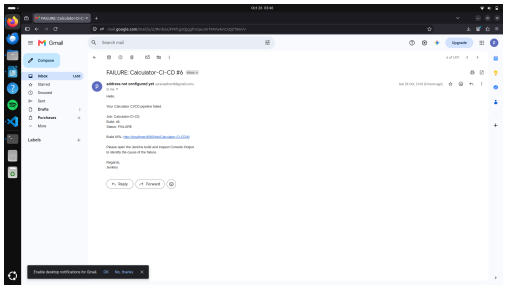
(b) JUnit results: 16 tests passed



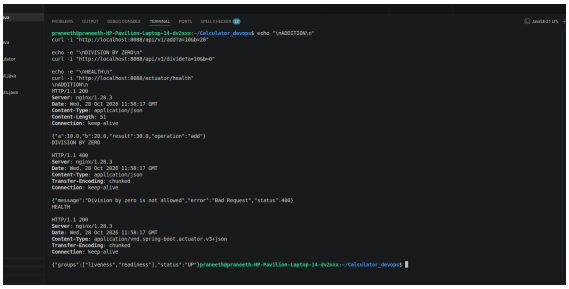
(c) `docker ps`: API healthy and Nginx running



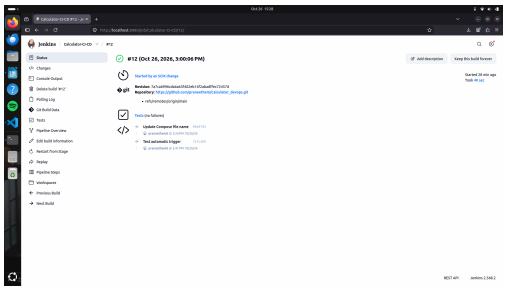
(d) Success email, build #11



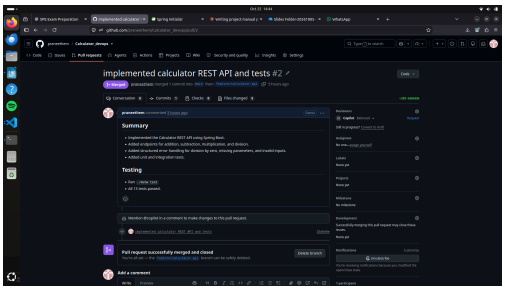
(e) Failure email, build #6



(f) API tests via Nginx: success, error, health check



(g) Successful automatic SCM-triggered build



(h) Merged feature-branch pull request

Figure 2: Evidence of CI/CD execution, testing, deployment, notifications, API behaviour, automatic triggering, and GitHub workflow.

7 Challenges Faced

One challenge was ensuring that pipeline failures were detected correctly and did not lead to deployment of an untested application. This was verified by deliberately introducing a failing test in Build #6, confirming that Jenkins stopped at the Test stage and sent a failure email. After restoring the test, a subsequent build succeeded. Another challenge was coordinating Docker image publishing and Compose deployment so that the updated application could be started and verified through its health endpoint. The deployment health check helped confirm that the application was running correctly behind Nginx. Finally, automatic triggering was tested through multiple GitHub commits, including after restarting Jenkins, to verify that changes were detected and processed without manually starting the pipeline.

8 Conclusion

The project demonstrates an end-to-end CI/CD pipeline integrating Jenkins, GitHub, Docker Hub, and Docker Compose. Jenkins automatically builds and tests the Calculator API, publishes its Docker image, deploys the application alongside Nginx, verifies application health, and sends email notifications. Build #6 was intentionally failed to verify failure detection and email notification, while subsequent successful builds demonstrated recovery. Automatic triggering through GitHub changes, including after restarting Jenkins, was also verified. Overall, the project demonstrates automated testing, containerized deployment, health verification, and pipeline failure handling.